



འབྲུག་རྒྱལ་ཁོངས་གཞི་རིག་སྡེ་ཁག་གི་སྡེ།

College of Science and Technology
Rinchending: Bhutan

WEB102
Server Application Fundamentals
(SS2026)

Practical 6 Report

Submitted By;

Name: Khamsum Yoezer

Student No.: 02250390

Programme: BE 1SWE

Date: 19/5/2026

RUB Wheel of Academic Law: Academic Dishonesty

Section H2 of the Royal University of Bhutan's *Wheel of Academic Law* provides the following definition of academic dishonesty:

Academic dishonesty may be defined as any attempt by a student to gain an unfair advantage in any assessment. It may be demonstrated by one of the following:

1. **Collusion:** the representation of a piece of unauthorised group work as the work of a single candidate.
2. **Commissioning:** submitting an assignment done by another person as the student's own work.
3. **Duplication:** the inclusion in coursework of material identical or substantially similar to material which has already been submitted for any other assessment within the University.
4. **False declaration:** making a false declaration to receive special consideration by an Examination Board or to obtain extensions to deadlines or exemption from work.
5. **Falsification of data:** presentation of data in laboratory reports, projects, etc., based on work purported to have been carried out by the student, which has been invented, altered or copied by the student.
6. **Plagiarism:** the unacknowledged use of another's work as if it were one's own.

Examples are:

- verbatim copying of another's work without acknowledgement.
- paraphrasing of another's work by simply changing a few words or altering the order of presentation, without acknowledgement.
- ideas or intellectual data in any form presented as one's own without acknowledging the source(s).
- making significant use of unattributed digital images such as graphs, tables, photographs, etc., taken from test books, articles, films, plays, handouts, the internet, or any other source, whether published or unpublished.
- submission of a piece of work which has previously been assessed for a different award or module or at a different institution as if it were new work.
- use of any material without prior permission of copyright from the appropriate authority or owner of the materials used".

Aim

The aim of this lab experiment is to develop and test a secure token-based authentication system using Node.js and JSON Web Tokens (JWT). The lab demonstrates how users can register, log in, receive authentication tokens, and access protected routes securely using middleware verification.

Theory

Token-Based Authentication

Token-based authentication is a modern authentication mechanism where the server generates a token after successful login. The client stores this token and sends it with future requests to access protected resources.

Unlike traditional session-based authentication, the server does not store session information. This makes token authentication scalable and suitable for distributed systems and APIs.

JSON Web Token (JWT)

A JSON Web Token (JWT) is a compact and secure way of transmitting information between two parties as a JSON object.

A JWT consists of three parts:

HEADER.PAYLOAD.SIGNATURE

Header

Contains information about the algorithm and token type.

Example:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload

Contains user data and claims.

Example:

```
{  
  "id": 1,  
  "email": "student@test.com"  
}
```

Signature

Ensures that the token has not been modified.

bcrypt Password Hashing

Passwords should never be stored as plain text. The bcrypt library hashes passwords into irreversible encrypted strings.

Example hashed password:

```
$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lLqq
```

This improves security because attackers cannot easily recover the original password even if the database is compromised.

Middleware in Express.js

Middleware functions are executed before the final route handler. In this project, middleware is used to verify JWT tokens before allowing access to protected routes.

If the token is valid:

- Access is granted.

If the token is invalid or missing:

- Access is denied with an error response.

Software and Tools Used

Software/Tool	Purpose
Visual Studio Code	Writing and editing code

Software/Tool	Purpose
Node.js	Running the server
Thunder Client	API testing
Postman	Alternative API testing tool
Express.js	Backend framework

Implementation Steps

Step 1: Create Project Folder

Commands used:

```
mkdir node-token-auth
```

```
cd node-token-auth
```

Step 2: Initialize Node.js Project

```
npm init -y
```

This creates the package.json file.

Step 3: Install Required Packages

```
npm install express jsonwebtoken bcryptjs dotenv
```

Step 4: Create Project Structure

Project structure:

```
node-token-auth/
```

```
|— server.js
```

```
|— .env
```

```
|— routes/
```

```
    |— auth.js
```

```
    |— protected.js
```

```
|— middleware/
```

```
    |— verifyToken.js
```

Step 5: Configure Environment Variables

Created .env file:

```
1 JWT_SECRET=supersecretkey123
2 PORT=3000
```

Step 6: Create Main Server File

Created server.js to:

- Initialize Express server
- Load middleware
- Connect routes
- Start server on port 3000

```
require('dotenv').config();
const express = require('express');

const authRoutes = require('./routes/auth');
const protectedRoutes = require('./routes/protected');

const app = express();
app.use(express.json());

// Routes
app.use('/auth', authRoutes);
app.use('/', protectedRoutes);

const PORT = process.env.PORT || 3000;
Tabnine | Edit | Test | Explain | Document
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
  console.log(`\nAvailable endpoints:`);
  console.log('  POST /auth/register - Create account');
  console.log('  POST /auth/login    - Login and get token');
  console.log('  GET  /profile       - Protected route (needs token)');
  console.log('  GET  /auth/users    - List all users (bonus homework)');
});
```

Step 7: Create JWT Verification Middleware

Created middleware/verifyToken.js.

Functions performed:

- Reads Authorization header
- Extracts Bearer token
- Verifies JWT signature
- Grants or denies access

```
const jwt = require('jsonwebtoken');

Tabnine | Edit | Test | Explain | Document
function verifyToken(req, res, next) {
  // Token comes in the header as: Authorization: Bearer <token>
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // grab the part after "Bearer "

  if (!token) {
    return res.status(401).json({ message: 'Access denied. No token provided.' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // attach decoded user info to the request object
    next();             // pass control to the actual route handler
  } catch (err) {
    return res.status(403).json({ message: 'Invalid or expired token.' });
  }
}

module.exports = verifyToken;
```

Step 8: Create Authentication Routes

Created routes/auth.js.

Implemented:

- User registration
- Password hashing
- User login
- JWT token generation


```

const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const router = express.Router();

// In-memory user store (no database needed for this demo)
const users = [];

// — POST /auth/register —————
Tabnine | Edit | Test | Explain | Document
router.post('/register', async (req, res) => {
  const { name, email, password } = req.body; // BONUS: accept 'name' field

  if (!email || !password) {
    return res.status(400).json({ message: 'Email and password are required.' });
  }

  // Check if user already exists
  const existingUser = users.find(u => u.email === email);
  if (existingUser) {
    return res.status(409).json({ message: 'User already exists.' });
  }

  // Hash the password before storing it (never store plain text!)
  const hashedPassword = await bcrypt.hash(password, 10);

  const newUser = {
    id: users.length + 1,
    name: name || null, // BONUS: store the name (null if not provided)
    email,
    password: hashedPassword,
  };
  users.push(newUser);

  res.status(201).json({ message: 'User registered successfully!' });
});

```

```

router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: 'Email and password are required.' });
  }

  // Find user by email
  const user = users.find(u => u.email === email);
  if (!user) {
    return res.status(401).json({ message: 'Invalid email or password.' });
  }

  // Compare the provided password with the stored hash
  const passwordMatch = await bcrypt.compare(password, user.password);
  if (!passwordMatch) {
    return res.status(401).json({ message: 'Invalid email or password.' });
  }

  // Create a JWT token – only safe, non-sensitive info goes in the payload
  const token = jwt.sign(
    { id: user.id, email: user.email }, // payload (never include password here!)
    process.env.JWT_SECRET,           // secret key from .env
    { expiresIn: '1d' }                // token expires in 1 day
  );

  res.json({ message: 'Login successful!', token });
});

// — GET /auth/users (BONUS homework route) —————
// No token needed – but we NEVER expose the hashed password
Tabnine | Edit | Test | Explain | Document
router.get('/users', (req, res) => {
  const safeUsers = users.map(({ id, name, email }) => ({ id, name, email }));
  res.json(safeUsers);
});

```

Step 9: Create Protected Route

Created routes/protected.js.

Implemented:

- /profile protected endpoint
- Middleware authentication check
- User information response

```
const express = require('express');
const verifyToken = require('../middleware/verifyToken');

const router = express.Router();

// GET /profile - protected: only accessible with a valid JWT token
Tabnine | Edit | Test | Explain | Document
router.get('/profile', verifyToken, (req, res) => {
  // req.user was attached by verifyToken middleware after decoding the JWT
  res.json({
    message: 'Welcome! You accessed a protected route.',
    user: req.user,
  });
});

module.exports = router;
```

Step 10: Run the Server

Command used:

node server.js

Server output:

Server running on http://localhost:3000

```
PS C:\Users\Dell\OneDrive\Desktop\WEB102_02250390_Practical06\node-token-auth> node server.js
◇ injected env (2) from .env // tip: # multiple files { path: ['.env.local', '.env'] }
Server running on http://localhost:3000

Available endpoints:
POST /auth/register - Create account
POST /auth/login    - Login and get token
GET  /profile       - Protected route (needs token)
GET  /auth/users     - List all users (bonus homework)
```

6. Testing and Results

Test 1: User Registration

Request

POST /auth/register

JSON Body

```
{
  "email": "student@test.com",
```

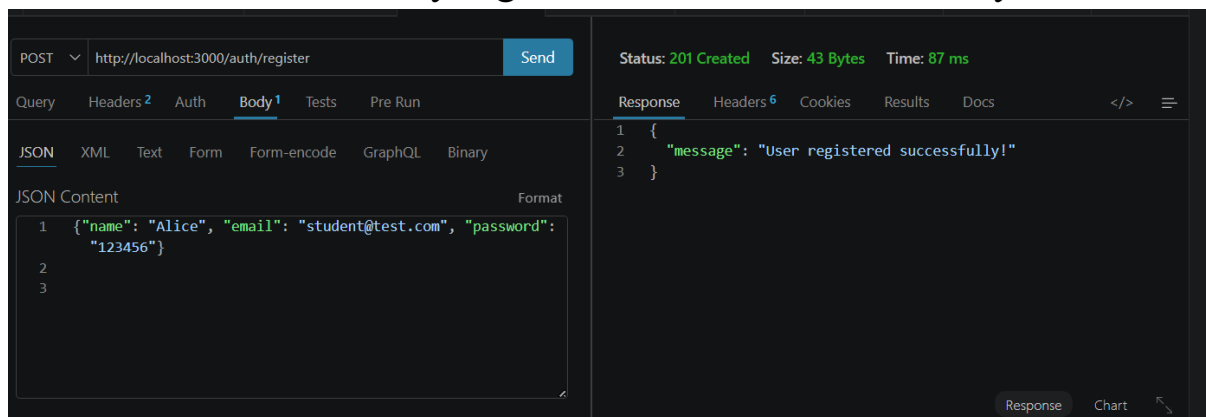
```
"password": "123456"
}
```

Expected Output

```
{
  "message": "User registered successfully!"
}
```

Result

The user was successfully registered and stored in memory.



Test 2: User Login

Request

POST /auth/login

JSON Body

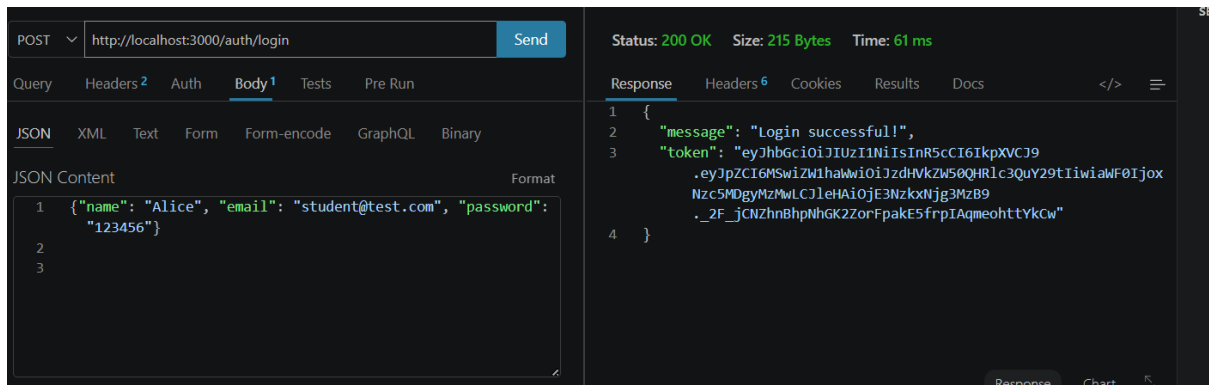
```
{
  "email": "student@test.com",
  "password": "123456"
}
```

Expected Output

```
{
  "message": "Login successful!",
  "token": "JWT_TOKEN"
}
```

Result

The server generated and returned a valid JWT token.



Test 3: Access Protected Route with Valid Token

Request

GET /profile

Header

Authorization: Bearer JWT_TOKEN

Expected Output

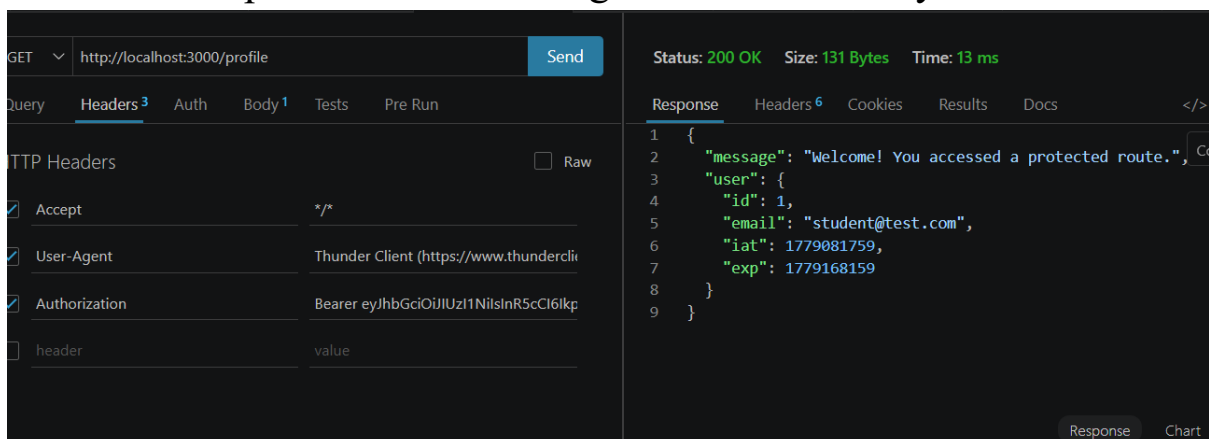
```

{
  "message": "Welcome! You accessed a protected route.",
  "user": {
    "id": 1,
    "email": "student@test.com"
  }
}

```

Result

Access to the protected route was granted successfully.



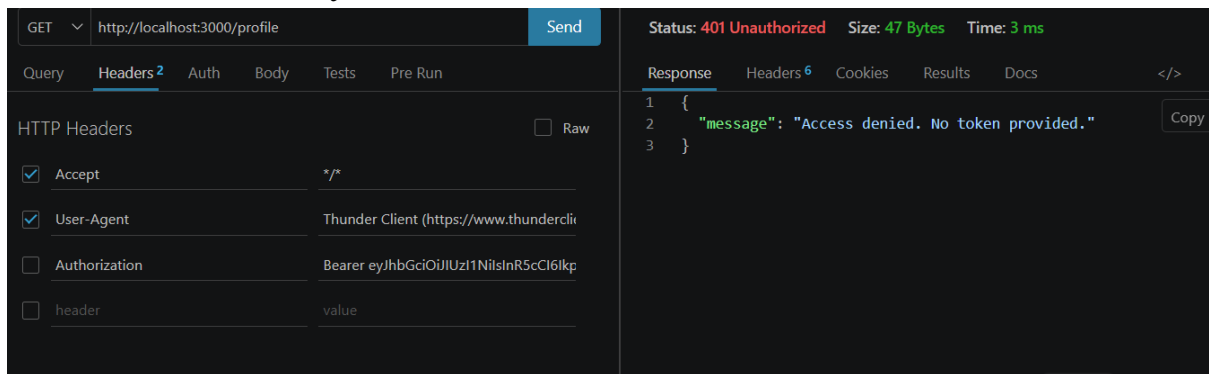
Test 4: Access Without Token

Expected Output

```
{
  "message": "Access denied. No token provided."
}
```

Result

The server correctly denied unauthorized access.



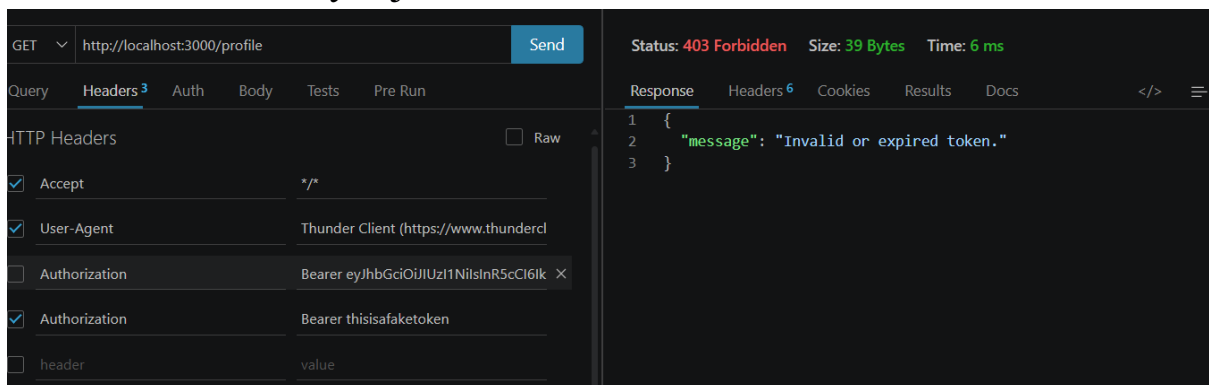
Test 5: Access with Invalid Token

Expected Output

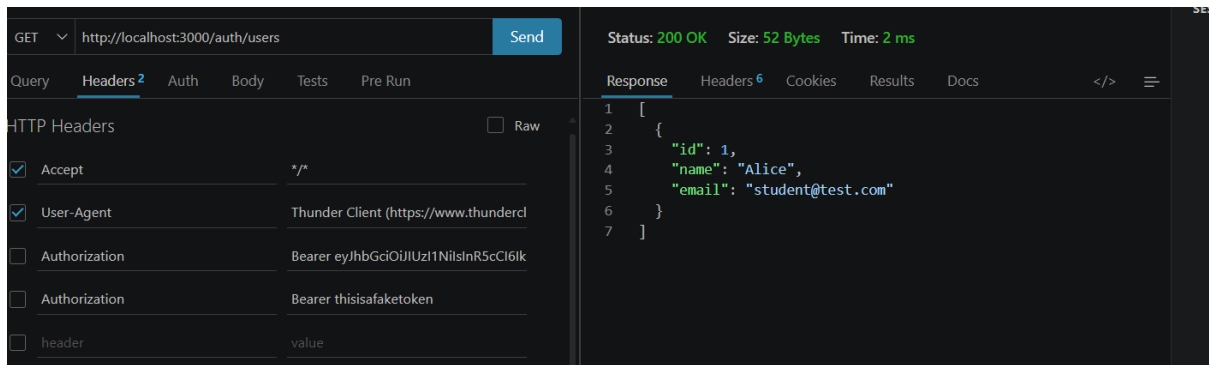
```
{
  "message": "Invalid or expired token."
}
```

Result

The server correctly rejected the fake token.



Test 6: User check



9. Conclusion

This lab successfully demonstrated the implementation of token-based authentication using JWT in Node.js. The project included user registration, secure password hashing, login authentication, JWT generation, middleware verification, and protected route access. The experiment also demonstrated how modern web applications use stateless authentication for improved scalability and security. Testing with Thunder Client confirmed that all endpoints functioned correctly, including successful authentication and rejection of invalid requests. Overall, the lab provided practical experience in backend authentication systems and secure API development.

10. References

- Auth0. (2024). *Introduction to JSON Web Tokens*.
<https://jwt.io/introduction>
- Express.js. (2024). *Express web framework documentation*.
<https://expressjs.com/>
- Mozilla Developer Network. (2024). *HTTP authentication*.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication>
- npm. (2024). *bcryptjs package documentation*.
<https://www.npmjs.com/package/bcryptjs>
- npm. (2024). *jsonwebtoken package documentation*.
<https://www.npmjs.com/package/jsonwebtoken>

- Node.js. (2024). *Node.js documentation*.
<https://nodejs.org/en/docs>